Aspectos destacados

De la detección a la acción: cerrando el bucle autonómico de una línea de fabricación autocoordinada con planificadores basados en reglas y en LLM con recuperación aumentada

Manuel Sánchez, Ernesto Exposito

- Nuestra arquitectura autonómica anterior detectaba condiciones, pero nunca planificaba ni actuaba.
- Sus alarmas son ráfagas escasas de toda la línea; con ellas ningún planificador recupera retraso.
- Un test por estación sobre tiempos de proceso hace que reparar en bucle cerrado compense.
- Un LLM de frontera que veta al test destaca donde las reparaciones falsas son caras.
- Una regla que veta ráfagas logra casi toda esa ganancia; con causa común, ambos pierden.

De la detección a la acción: cerrando el bucle autonómico de una línea de fabricación autocoordinada con planificadores basados en reglas y en LLM con recuperación aumentada

Manuel Sánchez[a,b,*], Ernesto Exposito[b]

[a] *Universidad Nacional Experimental del Táchira, San Cristóbal, Venezuela*
[b] *Univ Pau & Pays Adour / E2S UPPA, Laboratoire d'informatique de l'Université de Pau et des Pays de l'Adour (LIUPPA), EA3000, 64000 Pau, Francia*

Resumen

Las arquitecturas autonómicas para la Industria 4.0 prometen una producción que se autorrepara, pero los sistemas publicados a menudo se quedan en la detección. Revisamos nuestra arquitectura anterior de fabricación autocoordinada, comprobamos que implementa Monitor y Analyze pero no Plan, Execute ni una base de conocimiento, portamos su simulador a un motor sin interfaz gráfica y con pruebas sobre el conjunto de datos de la línea de producción de Bosch, y cerramos el bucle con un espacio de acciones construido sobre las estaciones gemelas paralelas de la línea, un validador simbólico, fallos sintéticos y un modelo de efectos explícito. Comparamos planificadores basados en reglas con planificadores basados en modelos de lenguaje de gran tamaño (LLM), con y sin estadísticas operativas recuperadas, sobre un modelo local de 7B (Qwen2.5) y un modelo de frontera (Claude Fable 5.1), con parámetros y prompts fijados en una traza de desarrollo, sobre 216 escenarios con fallo de tres trazas reservadas y 360 de cinco trazas generadas después de congelar el diseño, midiendo el retraso neto recuperado en las 24 h siguientes al fallo. La capa Analyze original es el cuello de botella: sus alarmas llegan en ráfagas de toda la línea, y con ella ningún planificador recupera más del 7 % del retraso. Con un test de cambio por estación sobre los tiempos de proceso, introducido después de observar ese fallo, un LLM de frontera que veta los candidatos del test recupera el 41 % (intervalo al

*Autor de correspondencia.
Email addresses: mbsanchez@unet.edu.ve (Manuel Sánchez), ernesto.exposito@univ-pau.fr (Ernesto Exposito)

Versión de lectura en español. El manuscrito que se envía a Computers in Industry es la versión en inglés.

95 %: 32–50 %) en las trazas reservadas y el 49 % (42–55 %) en las nuevas, frente al 20 % y el 27 % de la mejor regla con su configuración por defecto; tres réplicas sin respuestas almacenadas dan el 42–43 %. Buena parte de la ganancia viene de no actuar sobre alarmas compartidas por muchas estaciones: una regla que veta esas alarmas recupera el 34 % y el 40 %, y el margen del LLM sobre ella es de 8–9 puntos. Lo que vale el veto depende de lo que cuesta una reparación falsa. Con reparaciones largas, la línea parada, congestión completa o fallos leves, el híbrido con LLM pierde mucho menos que cualquier regla; con fallos graves, reparaciones cortas o sin congestión, todos los planificadores quedan igualados; y cuando una misma causa degrada cuatro estaciones a la vez, el veto descarta fallos reales y el híbrido queda por detrás de la regla simple. Un primer diseño de evaluación, con el fallo inyectado en un instante fijo dentro de la ráfaga de arranque del detector, había producido ganancias aparentes que desaparecieron al corregir el diseño.

Keywords: Computación autonómica, MAPE-K, Industria 4.0, Modelos de lenguaje de gran tamaño, Generación aumentada por recuperación, Gestión de fallos

---

1. Introducción

La computación autonómica propone sistemas que se gestionan a sí mismos mediante un bucle Monitor–Analyze–Plan–Execute sobre una base de conocimiento compartida (MAPE-K) [1, 2]. En la fabricación este bucle resulta atractivo porque las líneas de producción se degradan continuamente y la supervisión humana no escala: una línea que se autorrepara detectaría una estación que se degrada y se reconfiguraría antes de que el retraso se acumule.

En un trabajo anterior propusimos una arquitectura para procesos de fabricación autocoordinados con propiedades self-* y la evaluamos con un simulador, SSACSimulator, sobre el conjunto de datos de la línea de producción de Bosch [3, 4]. Al revisar ese código, comprobamos que solo implementa las fases Monitor y Analyze. Cuando uno de sus tres detectores se dispara, el simulador escribe *"launching the autonomous cycle for reconfiguration"* en un panel de registro; no se produce ningún plan y no se ejecuta nada. El bucle se anunciaba, pero no se cerraba.

Este artículo lo cierra y se pregunta qué hace falta realmente para cerrarlo. Dos desarrollos hacen oportuna la pregunta. Primero, los modelos de lenguaje de gran tamaño (LLM) se proponen cada vez más como planifica-

dores para la toma de decisiones industriales, desde el control de sistemas de producción modulares [5] hasta la planificación y programación avanzadas [6]. Segundo, la generación aumentada por recuperación (RAG) [7] es una candidata natural para el componente Knowledge de MAPE-K: en lugar de una ontología construida a mano, el planificador recupera el historial operativo que necesita. Si alguno de los dos ayuda, frente a qué baseline y con qué capa Analyze, es una cuestión empírica.

Nuestras contribuciones son:

1. Un port fiel, sin interfaz gráfica y con pruebas del simulador original [3], y un análisis de su capa Analyze sobre los datos publicados: un detector es inalcanzable, otro se ejercita con el 0,004 % de los eventos y el tercero emite ráfagas de alarmas de toda la línea, concentradas al inicio de cada traza, que en el día siguiente a un fallo nombran a la estación averiada solo en la mitad de los escenarios (secciones 3 y 7.1).
2. Las fases Plan, Execute y Knowledge que faltaban, y una capa Analyze por estación sobre los tiempos de proceso: un espacio de acciones basado en las estaciones "gemelas" paralelas de la línea de Bosch, un validador simbólico, inyección de fallos con tres formas y un modelo de efectos explícito y parametrizado (sección 4).
3. Una comparación de planificadores basados en reglas con planificadores LLM con y sin conocimiento recuperado, y con un híbrido en el que un test estadístico propone y el LLM veta, sobre un modelo local de 7B y un modelo de frontera, con las dos capas Analyze; todos los parámetros y prompts se eligen en una traza de desarrollo, y la evaluación usa 216 escenarios con fallo de tres trazas reservadas y 360 de cinco trazas generadas después de congelar el diseño (secciones 5–7).
4. Evidencia de que la capa Analyze fija lo que cualquier planificador puede recuperar y el planificador cuánto de ello se recupera; de que, con alarmas útiles, un LLM capaz aporta más como veto sobre un test estadístico, al evitar reparaciones que cuestan más de lo que arreglan; de que una regla simple que veta las ráfagas de alarmas reproduce la mayor parte de esa ganancia; de que el veto vale mucho donde las reparaciones falsas son caras, poco donde son baratas, y perjudica con fallos de causa común; y de que un inicio de fallo fijo puede fabricar conclusiones que desaparecen con un diseño más sólido (secciones 7 y 8).

Todo el código, los scripts de procesamiento de datos, los prompts, cada decisión de los modelos con la razón que dieron y todos los resultados están disponibles (véase «Disponibilidad de datos y código» al final del artículo).

2. Antecedentes y trabajo relacionado

Computación autonómica y sistemas autoadaptativos. El modelo de referencia MAPE-K [1] estructura un sistema autogestionado como un bucle de realimentación en torno a un recurso gestionado. La ingeniería de esos bucles es el objeto de la literatura sobre sistemas autoadaptativos [2], que subraya que la fase Analyze debe separar el cambio real del ruido antes de planificar cualquier adaptación. Recientemente se han estudiado los LLM como componentes de esos bucles: una revisión del estado del arte y hoja de ruta organiza sus usos potenciales en torno a las funciones de MAPE-K [8, 9]; se han integrado agentes basados en LLM en sistemas multiagente basados en MAPE-K [10]; y MAPER extiende MAPE-K con un componente de razonamiento basado en LLM para manejar situaciones imprevistas en tiempo de ejecución [11]. Nuestro estudio se diferencia en que sitúa un LLM dentro del bucle de un sistema industrial cuyo detector primero heredamos sin cambios y después sustituimos, y en que lo mide frente a baselines sin LLM ajustados, sobre datos reservados y sobre datos generados después de congelar el diseño.

Propiedades self- en la fabricación.* Nuestro trabajo previo integró sistemas multiagente, orientación a servicios y computación autonómica para la coordinación de procesos en la Industria 4.0 [3, 4, 12]. El presente artículo reexamina empíricamente esa arquitectura en lugar de proponer una nueva.

Gestión de alarmas y detección de cambios. En las industrias de proceso, las normas de gestión de alarmas exigen racionalizar las alarmas, de modo que cada una tenga sentido y requiera una respuesta, antes de pedir a los operadores que actúen sobre ellas [13]; un informe técnico aplica los mismos principios a la fabricación por lotes y discreta [14], el contexto de una línea como la de Bosch. Un controlador autonómico que actúa sobre alarmas se enfrenta al mismo requisito, sin un operador que absorba el ruido. Detectar un cambio en la tasa de un flujo de eventos o en la distribución de una medida es un problema clásico [15]; nuestros planificadores basados en reglas y nuestro detector por estación son instancias sencillas de él.

LLM y recuperación en la toma de decisiones industriales. RAG condiciona un modelo de lenguaje con documentos recuperados para cada consulta [7]. En la fabricación, se han combinado agentes LLM con gemelos digitales para planificar y controlar la producción modular [5] y se han aplicado a la planificación y programación avanzadas [6], y una revisión reciente de los LLM para el diagnóstico de fallos en la fabricación inteligente cubre,

entre otras funciones, la fusión de conocimiento industrial con recuperación aumentada [16], y otra revisa cómo se mantiene a las personas en el bucle cuando se usan LLM en la fabricación inteligente [17]. Aquí, el conocimiento recuperado son estadísticas operativas estructuradas, calculadas por código a partir del propio historial del controlador, no documentos; mantenemos el término RAG para el paso de recuperación e indicamos exactamente qué se recupera (sección 5).

3. El sistema original, revisado

3.1. Simulador y datos

SSACSimulator es un simulador de eventos discretos en Qt/C++. Un modelo de proceso, derivado del conjunto de datos Bosch Production Line Performance [18], describe 52 estaciones en cuatro sublíneas más los nodos de inicio y fin, 285 transiciones (84 de ellas flujos principales), un tiempo de ciclo de 6 420 min y 1 183 165 casos. Cuatro trazas de eventos de 100 000 transiciones cada una, preparadas para el estudio original [3], reproducen movimientos de piezas registrados; abarcan entre 54 y 168 horas de producción. Para la validación extrajimos cinco trazas más de los mismos ficheros en bruto, después de congelar el diseño (sección 6). Todas las trazas proceden del conjunto de prueba del conjunto de datos: sus 1 183 748 piezas, menos 583 sin ninguna marca temporal, dan los 1 183 165 casos del modelo de proceso. Los eventos de todas esas piezas se ordenan por tiempo y se cortan en porciones consecutivas de 100 000, de modo que cada traza es un periodo continuo de producción que contiene todas las piezas del conjunto de prueba activas en él, pero no las del conjunto de entrenamiento, que el estudio original no usó. Las marcas temporales están anonimizadas; como en el estudio original, una unidad del conjunto de datos se toma como 600 min, de modo que su resolución de 0,01 unidades da la granularidad de 6 min de las trazas. La producción se registra por lotes: 62 de las 157 horas de la traza 100m–200m no contienen ningún evento.

3.2. Port sin interfaz gráfica

Portamos a Python el núcleo de simulación (unas 1 440 líneas de lógica en C++, sin contar la interfaz gráfica). El port reproduce el comportamiento del original tal como se lee en su código fuente, incluidos detalles que cambian los resultados: umbrales calculados en precisión simple, un límite de carga de 100 000 eventos y una regla permisiva para las filas mal formadas. Un camino rápido avanza solo las piezas que pueden salir del sistema; el bucle original por tick se conserva como oráculo de pruebas, y una prueba de

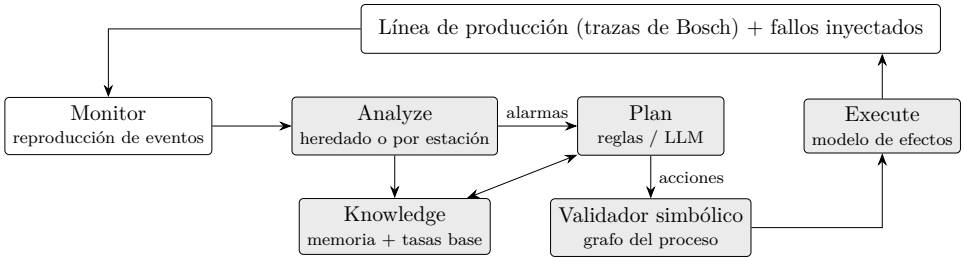


Figura 1: El bucle MAPE-K cerrado. En blanco: presente en el sistema original [3]. En gris: añadido o sustituido en este trabajo. La capa Analyze es el detector heredado o el detector por estación de la sección 4.4.

regresión comprueba que ambos producen detecciones, decisiones y tiempos de vida de pieza idénticos. La paridad es con el original tal como se lee en su código fuente, no con el binario Qt compilado. Una ejecución tarda entre 0,2 y 0,4 s.

3.3. Lo que hace realmente la capa Analyze

El original implementa tres detectores: una estación cuyo tiempo medio de transición supera 1,3 veces su referencia; más del 10 % de las estaciones marcadas dentro de un mismo lote de eventos simultáneos; y un tiempo de vida medio de las piezas que supera 1,1 veces el tiempo de ciclo. Solo se dispara el segundo. El detector de fallos predichos depende de una etiqueta de un clasificador que es positiva en 15 de 400 000 eventos en el conjunto de las trazas (0,004 %). El detector global es inalcanzable: el tiempo de vida medio de las piezas (636 min) está un orden de magnitud por debajo de su umbral (7 062 min). Por último, 45 de las 285 transiciones tienen un tiempo de referencia cero, de modo que cualquier duración positiva marca su estación. La sección 7.1 muestra qué ve el único detector activo cuando se inyectan fallos.

Plan, Execute y Knowledge están ausentes. Ese es el punto de partida del resto del artículo.

4. Cerrar el bucle

La figura 1 resume el bucle cerrado. Los componentes añadidos en este trabajo aparecen sombreados.

4.1. Estaciones gemelas y espacio de acciones

La línea de Bosch está formada por pares de estaciones paralelas (y algunos tríos) que comparten predecesoras y sucesoras y se reparten la carga casi por igual; por ejemplo, en sus flujos principales las estaciones L0_S2 y L0_S3 reciben 339 345 y 334 054 casos. Identificamos las gemelas automáticamente a partir del solapamiento de sus conjuntos de predecesoras y sucesoras en los flujos principales: 29 de las 52 estaciones tienen una. Desviar una pieza de una estación a su gemela mantiene su trayectoria coherente con el grafo del proceso. La equivalencia estructural no garantiza un equipamiento o una capacidad equivalentes, que el conjunto de datos anonimizado no describe; una planta tendría que confirmar cada gemela. El espacio de acciones es REROUTE(S, T, f), que desvía una fracción f de las piezas que entran en S a su gemela T; RESTORE(S); MAINTENANCE(S, d), que repara S durante d minutos; y NOOP.

4.2. Validador simbólico

Toda acción, de cualquier planificador, se comprueba frente al grafo del proceso y a las intervenciones en curso antes de ejecutarse: las estaciones deben existir, el destino de un desvío debe ser una gemela estructural, los desvíos no deben encadenarse y una estación no puede repararse dos veces a la vez. El validador nunca juzga si una acción es una buena idea, solo si es admisible.

4.3. Modelo de efectos

La simulación está guiada por la traza: los instantes de inicio proceden de los datos registrados y no se desplazan; los fallos y las acciones actúan sobre las duraciones de las transiciones, y el tiempo de vida de una pieza es la suma de las duraciones de sus transiciones. Por tanto, el retraso no se propaga en forma de colas; el modelo estima el tiempo perdido por pieza. Una "transición" de Bosch es el tiempo entre dos marcas de estación y mide sobre todo espera: su mediana es 0 min y su percentil 90 es de 18–30 min, pero su percentil 99 llega a 4 100–5 900 min. Por ello los efectos son *aditivos* y proporcionales al tiempo de proceso de cada estación τ_S (la mediana de sus transiciones de hasta 60 min), nunca multiplicativos sobre una transición que incluye días de almacenamiento. τ_S vale 6 o 12 min en las ocho estaciones donde se inyectan fallos, idéntico en todas las trazas. El modelo de efectos es:

- **Fallo** de factor k en S: $+(k(t)-1)\,\tau_S$ en cada transición que sale de S, donde $k(t)$ es un *escalón* (k desde t_0), *intermitente* (k durante 60 min, 1

durante 120 min, y así sucesivamente) o una *deriva* (crece linealmente de 1 a k en 600 min).

- **Desvío**: una fracción determinista de las piezas (seleccionadas mediante un hash del identificador del caso) va a T, que añade entonces $\alpha\, f\, \tau_T$ por la carga extra.
- **Mantenimiento**: los fallos de S ya activos cuando empieza la reparación se eliminan cuando termina; un fallo que empieza durante una reparación no se elimina. Un fallo intermitente cuenta como activo desde su inicio, también en sus fases apagadas. Durante la reparación, en modo *contingencia* (por defecto) una capacidad de respaldo fuera del modelo procesa las piezas, añadiendo $0{,}5\,\tau_S$; en modo *parada* las piezas esperan a que termine la reparación. Reparar una estación sana cuesta y no arregla nada, y eso es lo que encarece las falsas alarmas.

Como el modelo es aditivo, el retraso en exceso de una ejecución se descompone exactamente en los minutos añadidos a cada transición, lo que permite medirlo en cualquier ventana temporal. El conjunto de datos está anonimizado y no proporciona capacidades, tiempos de reparación ni fallos etiquetados, de modo que todos los fallos son sintéticos y todo parámetro que no pudimos fundamentar en los datos (severidad k, tiempo de reparación d, congestión α, modo de mantenimiento) se varía en un análisis de sensibilidad (sección 7.2).

4.4. Una capa Analyze por estación sobre los tiempos de proceso

La sección 7.1 muestra que el detector heredado informa de los fallos inyectados pocas veces y solo dentro de ráfagas de toda la línea. Su test por estación compara medias de transiciones completas, dominadas por horas o días de almacenamiento, de modo que un fallo que añade minutos de proceso apenas las mueve. Por ello añadimos una capa Analyze alternativa que mira solo el régimen de proceso: las transiciones que salen de una estación y duran como máximo 60 min. Para cada estación, de forma causal, toma como referencia el percentil 75 de esas transiciones en las 24 h anteriores y la tasa p_0 con la que lo superan (con suavizado de Laplace, porque las marcas de tiempo tienen una granularidad de 6 min); entre las últimas N transiciones de proceso de la estación, cuenta las que superan la referencia y marca la estación cuando ese recuento es improbable bajo una Binomial(N, p_0) al nivel α. No se marca nada durante un calentamiento de 8 h; las estaciones marcadas en un mismo lote de eventos simultáneos se comunican en una sola alarma, y una estación no vuelve a marcarse dentro de un periodo refractario.

N, α y el periodo refractario se eligieron en la traza de desarrollo (sección 6): $N = 10$, $\alpha = 10^{-3}$ y 6 min. Con esta capa, las alarmas de estación heredadas ya no se pasan a los planificadores. Los planificadores no cambian.

5. Planificadores

Todos los planificadores reciben, para cada alarma, la alarma y el estado de sus propias intervenciones anteriores. Ninguno ve los fallos inyectados. Todos responden del mismo modo ante una estación que consideran averiada: desviar a una gemela libre si existe y, después, reparar. Todos los planificadores, incluidos los LLM, dejan de actuar 240 min sobre una estación en la que acaban de actuar.

5.1. Planificadores basados en reglas

- **Ingenuo**: actúa sobre toda estación nombrada en una alarma.
- **Calibrado**: actúa cuando el número de alarmas de una estación en una ventana de 120 min es improbable (cola de Poisson, $\alpha = 10^{-3}$, al menos 3 alarmas) bajo su tasa base, aprendida en una traza sin fallos *distinta*.
- **Adaptativo**: el mismo test, frente a la tasa observada en las 24 h anteriores de la misma traza, de forma causal, tras un calentamiento de 8 h.
- **Adaptativo + veto por carga** (control): la regla adaptativa, ignorando las estaciones que reciben carga que el propio controlador desvió hacia ellas.
- **Solo gemelas** (control): actúa sobre toda estación marcada que tenga una gemela libre e ignora las demás.
- **Veto por ráfaga** (control): la regla adaptativa, que rechaza los candidatos de una alarma que nombra al menos m estaciones, o de cualquier alarma mientras al menos m estaciones distintas han alarmado en los últimos W min; con o sin veto por carga. Como en el híbrido, una estación rechazada no se reexamina durante 120 min. m y W se eligieron en la traza de desarrollo (sección 6): con veto por carga, alarmas de al menos 3 estaciones; sin él, al menos 8 estaciones alarmando en 120 min.

- **Veto aleatorio** (control): la regla adaptativa, que rechaza cada candidato al azar con la tasa de veto por candidato del híbrido en las trazas reservadas (70 %), con la misma suspensión de 120 min; cinco semillas.

Los controles no son métodos propuestos; implementan como reglas simples comportamientos observados en planificadores LLM, de modo que la contribución de un LLM se mide más allá de ellos. El veto por ráfaga es la razón que el híbrido sobre Fable 5.1 da con más frecuencia para sus vetos (sección 7.7). La ventana, el nivel y la historia por defecto se fijaron antes de cualquier evaluación; además los reajustamos en la traza de desarrollo (sección 6).

5.2. Planificadores LLM

Para cada alarma con estaciones accionables, el LLM debe devolver, para cada una de ellas, un veredicto (*degradada*, *normal*, *autoinducida* o *evidencia insuficiente*) seguido de un tipo de acción (ninguna, reparar, o desviar y reparar). La salida está restringida por un esquema JSON cuyos nombres de estación se limitan a las accionables, de modo que el decodificador no puede inventar una estación. El código elige la gemela, descarta toda acción incoherente con el propio veredicto del modelo y pasa el resto al validador.

- **C1** recibe la alarma, las estaciones accionables, si cada una tiene una gemela libre y las intervenciones en curso: la misma información estructural que las reglas.

- **C2 (RAG)** recibe además conocimiento recuperado de la memoria operativa del controlador: para cada estación, las alarmas observadas en las últimas 2 y 24 h frente al número que predice su tasa base sin fallos, si el controlador le está desviando carga, si sus alarmas bajaron tras reparaciones anteriores y sus registros más recientes. La recuperación se indexa por estación y se ordena por recencia; todas las estadísticas las calcula el código.

- **Híbrido**: el test adaptativo de detección de cambios propone las estaciones; el LLM, con el conocimiento de C2, confirma o veta y elige la acción. Una estación vetada no se vuelve a examinar durante 120 min. El prompt indica al modelo que el test suele acertar pero no ve el contexto, y da como ejemplos la carga que el controlador desvió hacia una estación, una reparación anterior que no redujo sus alarmas y *"a burst of alarms shared by many stations at once"* (una ráfaga de alarmas compartida por muchas estaciones a la vez).

El prompt de sistema describe cómo la capa Analyze emite las alarmas. La versión v2 del prompt describe el detector heredado y se fijó en la traza de desarrollo antes de cualquier ejecución en las trazas reservadas; la v3, usada con el detector por estación, solo difiere de la v2 en ese párrafo, y se escribió después de las ejecuciones reservadas con el detector heredado y se probó en la traza de desarrollo solo con el modelo de 7B (sección 6). La tabla 8 mide qué cambian ese párrafo y el ejemplo de las ráfagas.

Modelos. Usamos Qwen2.5-7B-Instruct [19], cuantizado a 4 bits y servido localmente con Ollama [20] a temperatura 0; y Claude Fable 5.1 (Anthropic), accedido mediante el Claude Agent SDK [21] con el esfuerzo en *low* (versión 0.2.163 del SDK). Cada llamada sustituye el prompt de sistema del entorno por el del planificador, no carga ninguna configuración ni fichero de memoria (`setting_sources` vacío, un directorio de trabajo temporal vacío), no expone herramientas ni servidores MCP e impone el esquema JSON del planificador como salida estructurada. El uso almacenado es coherente con ello: una llamada ocupa una mediana de 2 431 tokens de entrada, para 4 042 caracteres de prompt de sistema, petición y esquema. Todas las respuestas proceden de Fable 5.1: en 19 187 de las 19 189 llamadas almacenadas, los tokens de salida que el SDK atribuye a Fable 5.1 coinciden con los de la respuesta; las breves llamadas internas del propio entorno (una mediana de 16 tokens de salida) no producen ninguna parte de ella. El modelo de frontera no expone un ajuste de temperatura, por lo que sus ejecuciones no son reproducibles bit a bit. Todas las peticiones y respuestas de ambos modelos se almacenan, indexadas por la petición completa. Las peticiones idénticas son frecuentes, porque antes del inicio del fallo un escenario produce los mismos estados que la ejecución sin fallo de su traza, y se sirven desde ese almacén: con el detector por estación, 2 403 de las 26 933 llamadas de C2 sobre Fable 5.1 fueron nuevas. Una llamada nueva tarda una mediana de 5–8 s con cualquiera de los dos modelos, poco frente a fallos que cuestan horas. Cada decisión es una única muestra del modelo; para medir la variabilidad entre ejecuciones repetimos tres veces la ejecución principal del híbrido, cada una con su propia clave en el almacén de respuestas (sección 7.6). Fable 5.1 se usó a través de una suscripción de Claude, con el identificador de modelo `claude-fable-5-1`, entre el 6 y el 17 de octubre de 2026. Un primer piloto de desarrollo se descartó porque el SDK pasaba al modelo las definiciones de herramientas de los conectores de la cuenta; después se restringió el entorno a ninguna herramienta y se añadió su identificador a la clave del almacén, de modo que no se reutilizó ninguna respuesta de ese piloto. Los prompts figuran en el Apéndice A.

6. Protocolo experimental

Escenarios. Se inyecta un fallo de factor $k = 3$ en una de ocho estaciones, cuatro con gemela (L0_S2, L0_S4, L0_S6, L3_S35) y cuatro sin ella (L0_S1, L0_S8, L3_S29, L3_S33), todas con tiempos de proceso observables en cada traza, con una de las tres formas de fallo y en uno de tres inicios por estación y forma. Los inicios se sortean uniformemente entre 24 h después del primer evento de la traza y 24 h antes del último, con un generador sembrado con el nombre del escenario. Cada ejecución se detiene 24 h después del inicio del fallo. Cada traza tiene además una ejecución sin fallos sobre toda su longitud. Las reparaciones duran 60 min, $\alpha = 0{,}5$, y el mantenimiento funciona en modo contingencia salvo que se indique lo contrario.

Trazas de desarrollo y reservadas. La primera traza (0–100m) es la de desarrollo; las otras tres están reservadas (216 escenarios con fallo). Solo en la traza de desarrollo elegimos los parámetros del detector por estación (27 configuraciones de $N \in \{10, 20, 40\}$, $\alpha \in \{10^{-2}, 10^{-3}, 10^{-4}\}$ y periodos refractarios de 6, 30 y 60 min, seleccionadas por la recuperación de la regla adaptativa con veto por carga, de modo que el detector se ajustó para una regla y no para los planificadores LLM), reajustamos las reglas (45 configuraciones de ventana, nivel e historia para cada regla adaptativa, 15 de ventana y nivel para la calibrada) y fijamos los prompts. Cada versión de prompt probada y sus resultados quedan registrados en el repositorio. Las reglas calibradas y C2 aprenden sus tasas base en una traza sin fallos distinta de aquella en la que se evalúan (100m–200m a partir de la traza de desarrollo; 200m–300m y 500m–600m, una a partir de la otra). Durante el desarrollo de los prompts, las ejecuciones de desarrollo tomaron sus tasas base de 100m–200m, que después queda reservada; esto no expone ninguna etiqueta de fallo.

Secuencia de decisiones. Las trazas reservadas no quedaron al margen del rediseño, y decimos cómo. El diseño de evaluación (inicios, regla de reparación, horizonte) se revisó después de una primera evaluación en las trazas reservadas (sección 7.8). Con el diseño revisado, las reglas con el detector heredado recuperaron como mucho el 4 % en las trazas reservadas y la traza de desarrollo no tenía ninguna alarma heredada después de sus primeras 24 h, de modo que todas las configuraciones de reglas puntuaban cero en ella; esto es lo que nos llevó a construir el detector por estación. Antes de cualquier ejecución con él, registramos su forma en el repositorio: el régimen de proceso, la referencia de 24 h, el test binomial, el calentamiento y un

periodo refractario de 60 min, con N y α por elegir en la traza de desarrollo. Después añadimos el periodo refractario a la rejilla, porque con 60 min una estación nunca puede alcanzar las tres alarmas en 120 min que exigen las reglas; se eligieron 6 min. El prompt v3 se escribió entonces y se probó en la traza de desarrollo solo con el modelo de 7B. Todos los resultados reservados con el detector por estación se obtuvieron con estas decisiones fijadas. C1 sobre Fable 5.1 se ejecutó cuando ya se conocían los resultados reservados del híbrido. La condición de fallos leves y las ablaciones de la tabla 8 se registraron antes de ejecutarlas, después de los resultados reservados principales. Lo mismo, más tarde, los controles del veto, las demás condiciones de sensibilidad, los fallos de causa común, las réplicas y C2 en las trazas de validación; los parámetros de los controles se eligieron en la traza de desarrollo. En el mismo registro figura una nota sobre el periodo refractario, añadida a posteriori.

Trazas de validación. Para probar el diseño congelado sobre datos que no informaron ninguna decisión, generamos cinco trazas más a partir del conjunto de datos en bruto [18] con un port a Python 3 de los scripts de preprocesamiento originales, que reproduce las cuatro trazas existentes fila a fila. Tomamos, en orden, las cinco primeras porciones no usadas de 100 000 eventos (300m–400m, 400m–500m, 600m–700m, 700m–800m y 800m–900m; de 104 a 177 h cada una), con reglas de exclusión registradas antes de generarlas; no se excluyó ninguna. Todo lo demás quedó congelado: diseño, estaciones, formas, tres inicios por estación y forma (360 escenarios con fallo), detector, prompts, modelos y almacén de respuestas. Las tasas base proceden de la traza de desarrollo. Los planificadores son las reglas con los valores por defecto y reajustados, los controles del veto, los dos híbridos y C2 sobre Fable 5.1; C1 no se ejecutó allí.

Métricas. Cada ejecución reproduce la misma traza, de modo que cada pieza se compara consigo misma en una ejecución de referencia sin fallos. La *recuperación a 24 h* de un planificador sobre un conjunto de escenarios es $1-\sum E_{24}(\text{planificador})/\sum E_{24}(\text{no hacer nada})$, donde E_{24} es el retraso añadido a las transiciones en las 24 h siguientes al inicio del fallo: 1 elimina el coste de los fallos; 0 no es mejor que no hacer nada; los valores negativos significan que las propias acciones del planificador cuestan más que los fallos. La recuperación es, por tanto, neta del coste de actuar. Como el modelo de efectos es aditivo, E_{24} se descompone exactamente en el retraso propio del fallo que queda (en transiciones no cubiertas por una reparación o un desvío) y el retraso causado por las acciones del planificador; presentamos esta

descomposición sobre los escenarios en los que el fallo cuesta algo dentro de la ventana (tabla 4). La ventana es cronológica: la producción se registra por lotes, así que una estación puede tener pocas transiciones en ella, o ninguna. Un planificador *actúa* en un escenario si repara la estación averiada en esas 24 h; las *reparaciones falsas* son reparaciones de otras estaciones en la misma ventana. El *coste de las falsas alarmas* es el retraso en exceso por hora en las ejecuciones sin fallos.

Estadística. El estimando principal es la diferencia de recuperación a 24 h entre dos planificadores, con un intervalo bootstrap al 95 % [22] que remuestrea los escenarios dentro de su traza (10 000 remuestreos). Como los escenarios de una misma traza comparten el ruido de alarmas y tres trazas son pocos grupos, presentamos también un bootstrap en dos niveles que remuestrea trazas y, dentro de ellas, escenarios, un bootstrap que remuestrea los 24 (reservadas) o 40 (validación) grupos de traza y estación averiada, porque las ventanas de 24 h de los escenarios de una misma traza se solapan y comparten decisiones, la diferencia dejando fuera cada traza y la diferencia en cada extracción ordenada de trazas con reemplazo (27 para tres trazas, 3 125 para cinco). Los intervalos de la recuperación de un solo planificador usan 4 000 remuestreos. Son comprobaciones de estabilidad sobre las trazas de que disponemos, no estimaciones de cómo se generalizaría el resultado a otros periodos o líneas. Presentamos el número de escenarios en los que un planificador pierde menos tiempo que el otro y la correlación biserial de rangos para muestras pareadas r como tamaño del efecto; las pruebas de rangos con signo de Wilcoxon sobre el retraso por escenario [23], con ajuste de Holm [24], tratan los escenarios como independientes y figuran solo en el repositorio. El desglose por forma de fallo es exploratorio.

7. Resultados

7.1. La capa Analyze heredada da demasiado poca evidencia

El detector de estaciones heredado emite sus alarmas en ráfagas que siguen los lotes de producción, no los fallos. Sin fallos, en la traza reservada 100m–200m emite 63 y 18 alarmas en sus dos primeros bloques de 12 h y como mucho 4 en cada uno de los doce siguientes; en 200m–300m, 66 en las primeras 12 h, ninguna en las 60 h siguientes, después 25, 15 y 16 y como mucho 5 por bloque a partir de ahí; en la traza de desarrollo, 30, 9 y luego ninguna. La tabla 1 muestra lo que informa en las 24 h siguientes a un fallo, sin planificador. Nombra a la estación averiada en la mitad de los escenarios reservados, con una mediana de 10,9 h después del inicio, 5,4

Tabla 1: Lo que cada capa Analyze informa sobre la estación averiada en las 24 h siguientes al inicio del fallo, sin planificador, en los 216 escenarios con fallo reservados, y en las mismas ventanas sin el fallo. Retraso: mediana del tiempo desde el inicio hasta la primera alarma que nombra a la estación averiada, en los escenarios detectados. Alarmas por hora y estaciones por alarma: ejecuciones sin fallos de las tres trazas reservadas. El control de umbral usa el mismo régimen de proceso y las mismas ventanas que el detector por estación, con un umbral fijo en lugar del test binomial.

Capa Analyze	Escenarios con alarma (con / sin fallo)	Alarmas por escenario (con / sin fallo)	Retraso (h)	Alarmas por h sin fallos	Estaciones por alarma
Detector heredado	108 / 52	5,4 / 1,2	10,9	0,6	7,6
Detector por estación	122 / 107	15,1 / 3,6	3,5	1,6	2,0
Control de umbral	134 / –	–	4,7	4,4	3,4

veces de media frente a 1,2 en la misma ventana sin el fallo, y cada alarma que la nombra incluye 6,5 estaciones de media, porque el detector solo se dispara cuando más del 10 % de las estaciones se marcan en un mismo lote. Compara tiempos medios de transición dominados por el almacenamiento, de modo que un fallo que añade 12 min de proceso apenas los mueve.

Con esta capa Analyze, ningún planificador recupera más del 7 % en 24 h (tabla 3, bloque superior): las reglas calibrada y adaptativas van de $-0{,}03$ a $-0{,}02$ (el control de solo gemelas y C1 sobre el modelo de 7B lo hacen mucho peor), C2 sobre Fable 5.1 llega a $+0{,}07$ $[-0{,}00; +0{,}14]$ y el híbrido sobre Fable 5.1, que veta casi todos los candidatos, a $+0{,}03$. Los planificadores tienen demasiado poco sobre lo que actuar.

7.2. Una capa Analyze por estación

El detector por estación no informa de la estación averiada más a menudo, en relación con su fondo, que el detector heredado: la razón entre alarmas con y sin el fallo es de 4,1 frente a 4,5, y nombra a una estación sin fallo en la mitad de las ventanas (tabla 1). Lo que cambia es cuándo y cómo. Informa de la estación averiada con una mediana de 3,5 h después del inicio en lugar de 10,9 h, y en alarmas que incluyen 2,3 estaciones en lugar de 6,5 (sección 7.1), de modo que un test de tasa sobre sus alarmas puede aislar la estación. El test binomial es lo que lo hace utilizable: el control de umbral, con el mismo régimen de proceso y las mismas ventanas, detecta algunos fallos más pero emite 4,4 alarmas por hora sin fallos en lugar de 1,6, y con él las reglas caen a entre $-0{,}39$ y $-0{,}35$. El nivel del test binomial no es una tasa de falsas alarmas: las transiciones consecutivas de una estación están correlacionadas por lote y producto, de modo que el $\alpha = 10^{-3}$ nominal por test sigue dando 1,6 alarmas por hora en una línea sin fallos. En la traza de desarrollo, todas las configuraciones del detector por estación nombran

Tabla 2: Planificadores basados en reglas con el detector por estación, trazas reservadas (216 escenarios con fallo): recuperación a 24 h. Punto central: $k = 3$, reparaciones de 60 min, $\alpha = 0{,}5$, modo contingencia. Filas de sensibilidad: todas las formas de fallo, un parámetro cambiado. Hiperparámetros por defecto.

	Ingenuo	Calibrado	Adaptativo	Solo gemelas
Todas las formas	−0,28	+0,13	+0,17	−0,20
escalón	−0,07	+0,23	+0,23	−0,07
intermitente	−1,12	−0,29	−0,27	−0,64
deriva	−0,10	+0,22	+0,30	−0,13
$\alpha = 0$	–	+0,37	+0,41	+0,22
$\alpha = 1$	–	−0,07	−0,03	−0,61
$k = 1{,}5$	–	−1,51	−1,47	−1,75
$k = 5$	–	+0,37	+0,41	+0,03
Reparación de 15 min	–	+0,54	+0,58	+0,20
Reparación de 240 min	–	−1,22	−1,22	−1,73
Modo parada	–	−2,48	−2,55	−0,81

a la estación averiada en 24 h en los 72 escenarios de desarrollo; la elegida permite a la regla adaptativa con veto por carga recuperar +0,84 allí.

La tabla 2 recoge los planificadores basados en reglas con este detector en las trazas reservadas y un análisis de sensibilidad de un factor cada vez sobre todas las formas de fallo. Actuar sobre las alarmas del detector compensa solo cuando los fallos son graves en relación con el coste de una reparación y una capacidad de respaldo absorbe el trabajo durante las reparaciones. Reducir a la mitad el factor del fallo ($k = 1{,}5$, lo que divide por cuatro el retraso que añade un fallo), cuadruplicar el tiempo de reparación (240 min) o parar la línea durante las reparaciones hace que todas las reglas sean mucho peores que no hacer nada, porque las reparaciones falsas cuestan entonces más que los fallos que arreglan; la congestión completa ($\alpha = 1$) anula el beneficio. Reparaciones más cortas, fallos más graves o la ausencia de congestión lo multiplican por más de dos. El planificador ingenuo, que actúa sobre toda alarma, es peor que no hacer nada en el punto central, y con fallos intermitentes lo son todas las reglas. Repetimos todas las condiciones con los híbridos LLM y los controles del veto (sección 7.6).

7.3. Comparación en las trazas reservadas

La tabla 3 y la figura 2 recogen todos los planificadores en las tres trazas reservadas. Con el detector por estación, las reglas recuperan entre el 13 y el 20 % con su configuración por defecto; reajustarlas en la traza de desarrollo

Tabla 3: Trazas reservadas (216 escenarios con fallo, 3 ejecuciones sin fallos), horizonte de 24 h. Recuperación: neta, con intervalo bootstrap al 95 % estratificado por traza. Actúa: escenarios en los que se repara la estación averiada en 24 h. Reparaciones falsas: reparaciones de otras estaciones en 24 h, por escenario. Coste de falsas alarmas: min-pieza/h en las ejecuciones sin fallos, media de las tres trazas.

Planificador	Modelo	Recuperación	Escalón	Interm.	Deriva	Actúa	Reparaciones falsas	Coste de falsas alarmas
Detector heredado								
Regla calibrada	–	−0,03 [−0,12; +0,04]	+0,04	−0,38	+0,04	62	4,4	108
Regla adaptativa	–	−0,03 [−0,14; +0,06]	+0,06	−0,44	+0,06	77	6,2	67
Adaptativa + veto por carga	–	−0,02 [−0,12; +0,07]	+0,08	−0,40	+0,06	75	5,9	64
Solo gemelas (control)	–	−0,35 [−0,45; −0,27]	−0,26	−0,76	−0,27	47	7,1	142
C1	Qwen 7B	−0,21 [−0,30; −0,12]	−0,13	−0,63	−0,09	49	6,4	136
C2 (RAG)	Qwen 7B	−0,15 [−0,27; −0,05]	−0,06	−0,73	+0,02	104	9,5	159
Híbrido	Qwen 7B	−0,04 [−0,10; +0,03]	−0,02	−0,25	+0,05	51	4,0	38
C2 (RAG)	Fable 5.1	+0,07 [−0,00; +0,14]	+0,13	−0,17	+0,12	54	3,9	58
Híbrido	Fable 5.1	+0,03 [−0,01; +0,08]	+0,09	−0,03	+0,00	24	0,9	7
Detector por estación								
Regla calibrada	–	+0,13 [+0,01; +0,24]	+0,23	−0,29	+0,22	101	5,5	107
Regla adaptativa	–	+0,17 [+0,05; +0,27]	+0,23	−0,27	+0,30	108	5,9	98
Adaptativa + veto por carga	–	+0,20 [+0,09; +0,30]	+0,26	−0,20	+0,33	108	5,5	89
Solo gemelas (control)	–	−0,20 [−0,30; −0,11]	−0,07	−0,64	−0,13	60	6,2	108
Veto por ráfaga (control)	–	+0,27 [+0,16; +0,36]	+0,29	−0,07	+0,40	94	4,2	66
Ráfaga + veto por carga (control)	–	+0,34 [+0,24; +0,43]	+0,32	+0,21	+0,41	91	2,8	41
C1	Qwen 7B	−0,05 [−0,17; +0,06]	+0,06	−0,55	+0,06	90	7,6	121
C2 (RAG)	Qwen 7B	−0,25 [−0,42; −0,10]	−0,05	−1,06	−0,08	117	14,0	203
Híbrido	Qwen 7B	+0,29 [+0,18; +0,38]	+0,32	−0,03	+0,41	106	5,2	74
C1	Fable 5.1	+0,08 [+0,04; +0,14]	+0,02	+0,10	+0,14	19	0,1	1
C2 (RAG)	Fable 5.1	+0,14 [+0,02; +0,24]	+0,23	−0,25	+0,22	105	7,1	100
Híbrido	Fable 5.1	**+0,41** [+0,32; +0,50]	+0,35	+0,29	+0,53	87	**1,8**	**25**

ayuda a la regla calibrada (+0,27) pero no a las adaptativas (+0,14 y +0,18). El planificador ingenuo es peor que no hacer nada con los dos detectores (−0,25 y −0,28). Los controles de veto por ráfaga superan a todas las demás reglas: +0,27, y +0,34 con el veto por carga, que además reduce a la mitad las reparaciones falsas de la regla adaptativa (2,8 frente a 5,9 por escenario). Vetar al azar no ayuda: con la tasa de veto del híbrido, la regla adaptativa recupera +0,15 de media en cinco semillas (de +0,09 a +0,19).

Con el detector por estación, los planificadores se separan. El híbrido sobre Fable 5.1 llega a +0,41, la mayor recuperación neta, con 1,8 reparaciones de estaciones sanas por escenario y un coste de 25 min-pieza/h sin fallos, frente a 5,5 y 89 de la regla adaptativa con veto por carga y 2,8 y 41 del veto por ráfaga con veto por carga. Actúa sobre la estación averiada en menos escenarios que las reglas (87 frente a 108 de 216). El mismo híbrido sobre el modelo de 7B llega a +0,29. Sin el test, los planificadores LLM lo hacen peor: C2 sobre Fable 5.1 llega a +0,14, por debajo de la regla, y C2 sobre el modelo de 7B a −0,25, con 14 reparaciones falsas por escenario. Sin conocimiento recuperado, Fable 5.1 (C1) actúa sobre la estación averiada en solo 19 escenarios y casi nunca sobre una sana; el modelo de 7B actúa de todos modos. Con fallos intermitentes, el híbrido sobre Fable 5.1 recupera

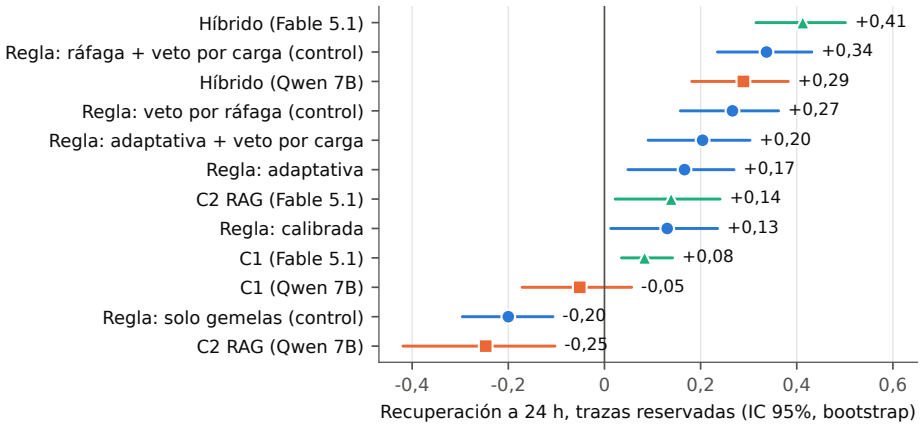


Figura 2: Recuperación neta a 24 h en las trazas reservadas con el detector por estación. Círculos: planificadores basados en reglas; cuadrados: Qwen2.5 7B; triángulos: Claude Fable 5.1.

+0,29, pero C1 sobre Fable 5.1, que casi nunca actúa, también supera allí a todas las reglas sin veto por ráfaga (+0,10); este desglose es exploratorio. La figura 3 sitúa cada planificador en el compromiso entre la recuperación con fallos y el coste de actuar sobre el ruido.

Escenarios sin coste y de dónde viene la recuperación. En 78 de los 216 escenarios reservados el fallo no cuesta nada en 24 h: en 73 la estación averiada no tiene ninguna transición en la ventana, porque la producción se registra por lotes, y en 5 todas las transiciones de un fallo intermitente caen en su fase apagada. Ahí, cualquier acción es puro coste. La tabla 4 restringe la comparación a los 138 escenarios con coste y descompone el retraso de cada planificador, como fracción del coste de no hacer nada, en el retraso propio del fallo que queda en la ventana y el retraso causado por las acciones del planificador (proceso en contingencia durante las reparaciones y congestión en las gemelas). La clasificación no cambia: el híbrido sobre Fable 5.1 recupera +0,44 y la regla adaptativa con veto por carga +0,27. Pero la descomposición muestra de dónde viene la diferencia. La regla elimina más parte del fallo (deja el 31 % del retraso del fallo, frente al 44 % del híbrido); el híbrido gana porque sus acciones cuestan un tercio (12 % frente a 42 %). C1 sobre Fable 5.1 marca el otro extremo: sus acciones casi no cuestan nada, pero deja el 91 % del fallo.

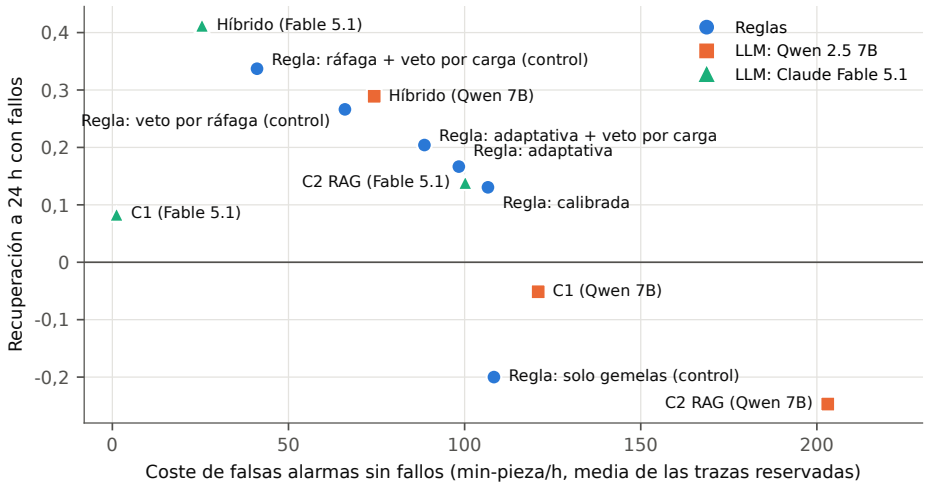


Figura 3: Recuperación neta a 24 h con fallos frente al coste de las falsas alarmas sin fallos, trazas reservadas, detector por estación.

7.4. Comparación estadística

La tabla 5 compara el híbrido sobre Fable 5.1 con los demás planificadores con el detector por estación. Su competidor más cercano es el control de veto por ráfaga con veto por carga: el híbrido recupera +0,08 más [+0,01; +0,14], pierde menos tiempo de producción en 124 escenarios y más en 19, y va por delante en cada traza reservada (+0,07, +0,12 y +0,05) y en las 27 extracciones ordenadas de tres trazas; por forma, el margen solo es claro con fallos de deriva. Frente a las reglas sin veto por ráfaga el margen es mayor: +0,21 [+0,14; +0,28] frente a la regla adaptativa con veto por carga (139 escenarios mejor, 18 peor, $r = 0{,}78$), positivo en cada traza (+0,23, +0,15 y +0,22), dejando fuera cualquier traza (+0,19 a +0,23), remuestreando las trazas como grupos ([+0,10; +0,28]) y remuestreando grupos de traza y estación ([+0,13; +0,29]); sobre los 138 escenarios con coste es de +0,17 [+0,11; +0,24]. Se mantiene también frente a las reglas reajustadas en la traza de desarrollo (+0,15 [+0,08; +0,22] frente a la mejor de ellas) y frente a la configuración de la rejilla que es la mejor en las propias trazas reservadas (+0,13 [+0,06; +0,20]), que ningún protocolo honesto podría seleccionar. Frente a la regla calibrada reajustada, el híbrido gana con fallos intermitentes (+0,25 [+0,12; +0,37]) y de deriva (+0,22 [+0,14; +0,32]), y ambos quedan igualados con fallos en escalón (+0,03 [−0,08; +0,15]). El híbrido sobre el modelo de 7B supera a la regla por defecto (+0,08, [+0,05; +0,13]

Tabla 4: Los 138 escenarios reservados en los que el fallo cuesta algo en 24 h, detector por estación. Fallo restante y coste de las acciones son fracciones del coste de no hacer nada; recuperación = 1− fallo restante − coste de las acciones. Recuperación con intervalo bootstrap al 95 % estratificado por traza.

Planificador	Modelo	Recuperación	Fallo restante	Coste de las acciones
Regla calibrada	–	+0,22 [+0,12; +0,32]	0,34	0,43
Regla adaptativa	–	+0,23 [+0,13; +0,33]	0,31	0,46
Adaptativa + veto por carga	–	+0,27 [+0,17; +0,36]	0,31	0,42
Solo gemelas (control)	–	−0,12 [−0,21; −0,03]	0,69	0,43
C1	Qwen 7B	+0,05 [−0,05; +0,14]	0,45	0,50
C2 (RAG)	Qwen 7B	−0,06 [−0,18; +0,05]	0,23	0,83
Híbrido	Qwen 7B	+0,35 [+0,25; +0,44]	0,31	0,34
C1	Fable 5.1	+0,08 [+0,04; +0,14]	0,91	0,01
C2 (RAG)	Fable 5.1	+0,24 [+0,14; +0,33]	0,33	0,44
Híbrido	Fable 5.1	**+0,44** [+0,35; +0,53]	0,44	**0,12**

remuestreando trazas), pero queda igualado con la mejor regla reajustada (+0,02 [−0,02; +0,07]) y con el control de veto por ráfaga con veto por carga (−0,05 [−0,11; +0,02]). C2 sobre Fable 5.1 queda por detrás de la regla (−0,07 [−0,17; −0,00] remuestreando trazas). Las pruebas de Wilcoxon, que tratan los escenarios como independientes aunque sus ventanas se solapan y comparten decisiones almacenadas, están en el repositorio; nos basamos en los intervalos.

Oportunidad, pérdidas y llamadas. Con una ventana más corta, de 8 h tras el inicio, la clasificación es la misma: el híbrido sobre Fable 5.1 recupera +0,36, el híbrido sobre el modelo de 7B +0,20, la regla adaptativa con veto por carga +0,11 y C2 sobre Fable 5.1 +0,09. El híbrido no actúa antes: cuando repara la estación averiada, su primera acción llega con una mediana de 345 min después del inicio, frente a 311 min de la regla y 218 min de la regla calibrada. Su ventaja no se paga con grandes pérdidas frente a no hacer nada: en su peor escenario añade 2 031 min-pieza al retraso de no hacer nada (el coste medio de no hacer nada es de 4 881 min-pieza por escenario), frente a 7 902 de la regla adaptativa con veto por carga. Sus mayores pérdidas frente a la regla vienen de vetar fallos reales: en los dos peores escenarios, fallos en escalón en 200m–300m, la regla eliminó alrededor del 60 % del fallo y el híbrido nada, una diferencia de 13 941 y 14 508 min-pieza. El híbrido llama al modelo unas 20 veces por escenario (mediana 22) y entre 0,2 y 0,3 veces por hora sin fallos; C2 lo llama entre 1,0 y 1,5 veces por hora. Cargar un coste fijo por cada reparación, como haría un presupuesto de mantenimiento, solo amplía la diferencia: con 240 min-pieza por reparación, el híbrido sigue recuperando +0,31 en las trazas reservadas y +0,35 en las de validación, mientras que la regla adaptativa con veto por carga cae a −0,09 y −0,19 y

Tabla 5: El híbrido sobre Claude Fable 5.1 frente a otros planificadores, detector por estación, pareado sobre los 216 escenarios con fallo reservados. Δ: diferencia de recuperación neta a 24 h (positiva a favor del híbrido sobre Fable 5.1), con intervalos bootstrap al 95 % que remuestrean los escenarios dentro de su traza y, de forma más conservadora, los 24 grupos de traza y estación averiada. r: correlación biserial de rangos para muestras pareadas. Reglas reajustadas: elegidas en la traza de desarrollo, salvo la fila marcada (la mejor en las trazas reservadas). El desglose por forma de fallo es exploratorio.

Frente a	Mejor / peor	r	Δ [IC 95 %, por traza]	[IC 95 %, por grupo]
Ráfaga + veto por carga (control)	124 / 19	0,64	+0,08 [+0,01; +0,14]	[+0,01; +0,14]
Veto por ráfaga (control)	132 / 16	0,74	+0,15 [+0,08; +0,21]	[+0,08; +0,23]
Adaptativa + veto por carga	139 / 18	0,78	+0,21 [+0,14; +0,28]	[+0,13; +0,29]
Regla adaptativa	144 / 14	0,81	+0,25 [+0,17; +0,32]	[+0,16; +0,34]
Regla calibrada	138 / 22	0,77	+0,28 [+0,20; +0,37]	[+0,19; +0,40]
Híbrido, Qwen 7B	133 / 25	0,68	+0,12 [+0,06; +0,19]	[+0,05; +0,20]
C2 (RAG), Fable 5.1	141 / 20	0,82	+0,27 [+0,20; +0,35]	[+0,19; +0,37]
Frente a reglas reajustadas				
Calibrada, reajustada	116 / 37	0,62	+0,15 [+0,08; +0,22]	[+0,07; +0,24]
Adaptativa + veto por carga, reajustada	143 / 18	0,78	+0,23 [+0,16; +0,30]	[+0,15; +0,32]
Adaptativa + veto por carga, mejor en reservadas	128 / 31	0,66	+0,13 [+0,06; +0,20]	[+0,06; +0,21]
Por forma de fallo, frente a adaptativa + veto por carga				
escalón	42 / 7	0,64	+0,09 [−0,02; +0,22]	[−0,03; +0,23]
intermitente	49 / 5	0,85	+0,49 [+0,35; +0,64]	[+0,34; +0,70]
deriva	48 / 6	0,83	+0,20 [+0,13; +0,27]	[+0,12; +0,30]
Por forma de fallo, frente a ráfaga + veto por carga				
escalón	38 / 8	0,57	+0,03 [−0,09; +0,15]	[−0,09; +0,16]
intermitente	40 / 9	0,47	+0,08 [−0,03; +0,19]	[−0,02; +0,21]
deriva	46 / 2	0,91	+0,12 [+0,06; +0,22]	[+0,07; +0,19]

la regla calibrada reajustada recupera +0,17 en las de validación.

7.5. Validación en trazas nuevas

La tabla 6 recoge las cinco trazas de validación, generadas después de congelar todas las decisiones de diseño (sección 6). El híbrido sobre Fable 5.1 recupera +0,49 [+0,42; +0,55]. La mejor regla vuelve a ser el control de veto por ráfaga con veto por carga, +0,40 [+0,32; +0,47], y el híbrido lo supera en +0,09 [+0,04; +0,15] ([+0,03; +0,20] remuestreando trazas como grupos, [+0,03; +0,16] con grupos de traza y estación), en cada una de las cinco trazas (de +0,05 a +0,29) y en las 3 125 extracciones ordenadas de cinco trazas; pierde menos tiempo en 212 escenarios y más en 75. Entre las reglas sin veto por ráfaga, la mejor es la calibrada reajustada en la traza de desarrollo, +0,37, y el margen del híbrido sobre ella es de +0,12 [+0,08; +0,16] (positivo en cada traza; [+0,07; +0,18] con grupos de traza y estación). El híbrido también va por delante en los 222 escenarios con coste (+0,55 frente a +0,47 y +0,46). El híbrido sobre el modelo de 7B no se replica: +0,23, por debajo de las dos versiones de la regla calibrada y de los dos controles de ráfaga. Sin el test, C2 sobre Fable 5.1 recupera +0,29 [+0,20; +0,36]: por delante de la regla adaptativa con veto por carga (+0,12 [+0,07; +0,17])

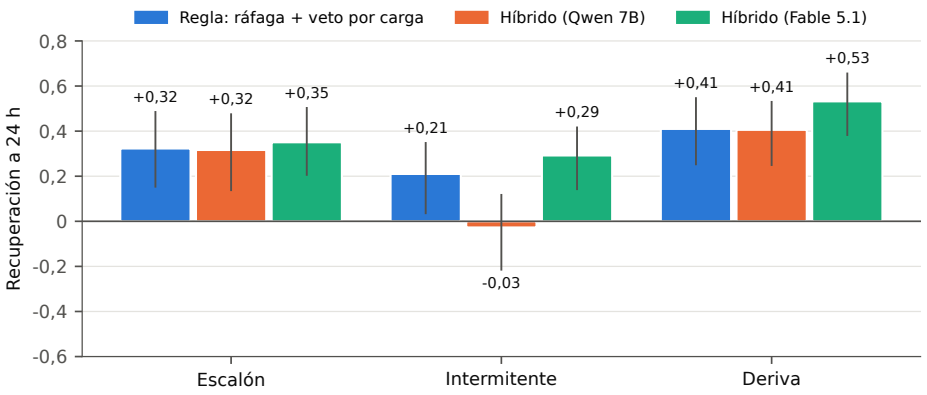


Figura 4: Recuperación neta a 24 h por forma de fallo en las trazas reservadas, detector por estación (72 escenarios por forma). Bigotes: intervalo bootstrap al 95 % estratificado por traza.

Tabla 6: Validación en cinco trazas nuevas (360 escenarios con fallo), todo congelado: recuperación neta a 24 h con intervalo bootstrap al 95 % estratificado por traza, y sobre los 222 escenarios con coste. Tasas base de la traza de desarrollo.

Planificador	Modelo	Recuperación	Con coste	Intermitente	Δ frente al híbrido (Fable 5.1)
Regla calibrada	–	+0,27 [+0,19; +0,35]	+0,40	−0,31	−0,21 [−0,26; −0,17]
reajustada	–	+0,37 [+0,30; +0,44]	+0,46	−0,05	−0,12 [−0,16; −0,08]
Regla adaptativa	–	+0,10 [−0,00; +0,19]	+0,29	−0,64	–
Adaptativa + veto por carga	–	+0,17 [+0,07; +0,25]	+0,34	−0,51	−0,32 [−0,40; −0,26]
reajustada	–	+0,19 [+0,09; +0,27]	+0,35	−0,49	−0,30 [−0,37; −0,24]
Solo gemelas (control)	–	−0,31 [−0,41; −0,23]	−0,15	−1,03	–
Veto por ráfaga (control)	–	+0,33 [+0,25; +0,40]	+0,42	−0,20	−0,16 [−0,22; −0,10]
Ráfaga + veto por carga (control)	–	+0,40 [+0,32; +0,47]	+0,47	+0,05	−0,09 [−0,15; −0,04]
C2 (RAG)	Fable 5.1	+0,29 [+0,20; +0,36]	+0,43	−0,27	−0,20 [−0,25; −0,16]
Híbrido	Qwen 7B	+0,23 [+0,14; +0,31]	+0,38	−0,35	−0,26 [−0,32; −0,19]
Híbrido	Fable 5.1	**+0,49** [+0,42; +0,55]	**+0,55**	+0,15	–

pero por detrás de la regla calibrada reajustada (−0,08 [−0,12; −0,05]), del control de ráfaga con veto por carga (−0,11 [−0,17; −0,05]) y del híbrido sobre el mismo modelo (−0,20 [−0,25; −0,16]).

7.6. Cuándo compensa el veto

La tabla 7 repite la comparación reservada en todas las condiciones del análisis de sensibilidad de las reglas y con fallos de causa común, con todos los planificadores congelados; el prompt sigue describiendo reparaciones de 60 min con capacidad de respaldo. Las condiciones se reparten en dos grupos. Donde una reparación falsa es cara (fallos leves, reparaciones de 240 min, línea parada durante las reparaciones, congestión completa), todas las reglas sin veto por ráfaga lo hacen mucho peor que no hacer nada o ganan poco,

Tabla 7: Sensibilidad de los planificadores, trazas reservadas, detector por estación: recuperación neta a 24 h. Un parámetro cambiado cada vez desde el punto central ($k = 3$, reparaciones de 60 min, $\alpha = 0{,}5$, modo contingencia); última fila: el mismo fallo, a la vez, en las cuatro estaciones con gemela o en las cuatro sin ella (54 escenarios). Reglas con su configuración por defecto salvo la calibrada reajustada. Δ: híbrido sobre Fable 5.1 menos el control de ráfaga con veto por carga, intervalo bootstrap al 95 % estratificado por traza.

Condición	Adaptativa + veto por carga	Calibrada, reajustada	Ráfaga + veto por carga	Híbrido, Qwen 7B	Híbrido, Fable 5.1	Δ [IC 95 %]
Central	+0,20	+0,27	+0,34	+0,29	**+0,41**	+0,08 [+0,01; +0,14]
Reparaciones falsas caras						
$k = 1{,}5$	−1,30	−0,79	−0,44	−0,95	**−0,07**	+0,37 [+0,29; +0,46]
Reparación de 240 min	−0,86	−0,81	−0,20	−0,90	**−0,09**	+0,11 [+0,02; +0,20]
Modo parada	−1,75	−1,40	−0,76	−1,86	**−0,56**	+0,20 [+0,11; +0,30]
$\alpha = 1$	−0,01	+0,13	+0,23	+0,18	**+0,36**	+0,13 [+0,06; +0,20]
Actuar es barato						
$k = 5$	+0,43	+0,43	+0,47	+0,47	**+0,50**	+0,02 [−0,04; +0,08]
Reparación de 15 min	+0,58	+0,53	+0,53	**+0,59**	+0,56	+0,03 [−0,01; +0,08]
$\alpha = 0$	**+0,46**	+0,43	+0,45	+0,40	**+0,46**	+0,01 [−0,05; +0,07]
Causa común	+0,53	+0,51	+0,36	**+0,55**	+0,43	+0,07 [−0,02; +0,15]

el veto por ráfaga limita el daño y el híbrido sobre Fable 5.1 lo limita más: recupera −0,07, −0,09, −0,56 y +0,36, por delante del control de ráfaga con veto por carga en +0,11 a +0,37 y de la mejor regla sin veto por ráfaga en +0,23 a +0,82. Donde actuar es barato (fallos graves, reparaciones de 15 min, sin congestión), los planificadores de la tabla 7 recuperan 0,4–0,6 y el híbrido queda igualado con la mejor regla (de −0,03 a +0,02, intervalos que incluyen el cero); con reparaciones de 15 min, el híbrido sobre el modelo de 7B y la regla adaptativa simple quedan ligeramente por delante.

Los fallos de causa común vuelven el veto contra sí mismo. Cuando un mismo fallo degrada cuatro estaciones a la vez, sus alarmas son, por construcción, ráfagas compartidas; el control de ráfaga las veta y cae a +0,36, y el híbrido sobre Fable 5.1 a +0,43, 0,10 por debajo de la regla adaptativa con veto por carga ([−0,17; −0,05]). El híbrido sobre el modelo de 7B, que rara vez usa el argumento de las ráfagas (sección 7.7), es el mejor planificador allí (+0,55). Son 54 escenarios en tres trazas, y el fallo es una construcción nuestra; pero muestran que el argumento que hace valioso el veto con fallos de una sola estación es erróneo cuando la causa sí es común.

Tres réplicas de la ejecución principal del híbrido, cada una con su propia clave en el almacén de respuestas para que no se reutilice ninguna respuesta de la ejecución original, recuperan +0,43, +0,42 y +0,42 (original +0,41) y siguen +0,21 a +0,22 por delante de la regla adaptativa con veto por carga; cada una necesitó unas 330 llamadas nuevas. La variación entre ejecuciones del modelo de frontera es pequeña frente a las diferencias que presentamos.

Tabla 8: Ablaciones del planificador híbrido sobre Claude Fable 5.1, trazas reservadas, recuperación neta a 24 h con intervalo bootstrap al 95 % estratificado por traza. Texto de alarmas: el párrafo del prompt que describe el detector (v2: heredado; v3: por estación). Pista de ráfagas: la mención, en el prompt del híbrido, de "a burst of alarms shared by many stations at once" entre las razones para dudar del test. Primera fila: la configuración de las tablas 3–5.

Detector	Texto de alarmas	Pista de ráfagas	Recuperación	Llamadas con veto	Vetos que citan ráfagas
Por estación	v3	sí	+0,41 [+0,32; +0,50]	2 833	91 %
Por estación	v2	sí	+0,41 [+0,32; +0,51]	2 601	88 %
Por estación	v3	no	+0,31 [+0,21; +0,41]	1 409	44 %
Heredado	v2	sí	+0,03 [−0,01; +0,08]	1 607	99 %
Heredado	v3	sí	+0,03 [−0,00; +0,07]	1 784	97 %

7.7. En qué se apoyan los vetos

El híbrido sobre Fable 5.1 es el planificador más conservador con el detector heredado y el mejor con el detector por estación. Las razones que da apenas cambian: codificadas con una regla explícita de palabras clave (*burst*, *line-wide*, *common cause*, *systemic*; regla y recuentos en el repositorio; una regla más amplia que añade palabras como *shared* y *upstream* da el 99 % y el 94 %), el 99 % de sus vetos argumenta que las alarmas llegan en ráfagas compartidas por muchas estaciones con el detector heredado (1 596 de las 1 607 llamadas con veto) y el 91 % con el detector por estación (2 587 de 2 833); contando respuestas distintas en lugar de llamadas, el 97 % y el 88 %. Además de las propias alarmas, hay dos cosas que podrían explicarlo: el párrafo del prompt que describe el detector cambia entre ambas condiciones, y el prompt del híbrido menciona esas ráfagas entre las razones para dudar del test. La tabla 8 las separa.

La descripción del detector no cambia la recuperación agregada: con cualquiera de los dos detectores, cambiar el párrafo de alarmas la deja igual (−0,00 [−0,05; +0,03] y +0,00 [−0,03; +0,03]). Sí cambia decisiones: con el detector por estación, el texto v3 pierde menos tiempo en 83 escenarios y más en 8, y las diferencias por traza (+0,03, −0,06, −0,00) se compensan; los intervalos no muestran que ambos textos sean equivalentes, solo que ninguno es mejor en conjunto. La estructura de las alarmas importa más: con el mismo prompt, el híbrido recupera +0,03 con las ráfagas del detector heredado y +0,41 con las alarmas por estación. La pista de las ráfagas ayuda: sin ella, el híbrido veta en la mitad de llamadas, cita ráfagas en el 44 % de sus vetos en lugar del 91 % y recupera 0,10 menos ([+0,07; +0,13], positiva en cada traza).

La tabla 9 muestra a qué responde el veto. El híbrido sobre Fable 5.1

Tabla 9: Proporción de los candidatos del test de cambio que veta cada híbrido, según el número de estaciones que nombra la alarma que dispara la llamada, desde el inicio del fallo; trazas reservadas, detector por estación. Entre paréntesis: número de candidatos. Recalculado reproduciendo las decisiones almacenadas.

	Fable 5.1		Qwen 7B	
Estaciones en la alarma	averiada	sanas	averiada	sanas
1	17 % (66)	40 % (329)	2 % (44)	17 % (298)
2	32 % (25)	64 % (319)	3 % (29)	11 % (199)
3	57 % (23)	93 % (314)	6 % (18)	17 % (266)
4	38 % (8)	91 % (172)	0 % (12)	8 % (165)
5 o más	88 % (17)	91 % (363)	26 % (19)	16 % (385)

veta un candidato con más frecuencia cuantas más estaciones nombra la alarma, como haría una regla de ráfagas; pero en todos los tamaños de alarma veta la estación averiada con menos frecuencia que una sana (17 % frente a 40 % en alarmas de una estación, 57 % frente a 93 % en alarmas de tres). Esa diferencia es lo que el control de ráfaga no puede reproducir. El híbrido sobre el modelo de 7B veta poco en cualquier tamaño y apenas separa las estaciones averiadas de las sanas; usa el argumento de las ráfagas en el 0,4 % de sus vetos con el detector heredado y en ninguno con el detector por estación según la regla de codificación estricta, pero en el 43 % con el detector heredado según la amplia, de modo que esa afirmación depende de la codificación.

7.8. Por qué importa el diseño de evaluación

En una primera fase de esta investigación, cada fallo se inyectaba 600 min después del primer evento de la traza, una reparación eliminaba cualquier fallo que empezara durante ella y la recuperación se medía sobre toda la traza. Con ese diseño el detector heredado parecía funcionar: las reglas adaptativas recuperaban el 52–54 %, C2 sobre Fable 5.1 el 58 % y el híbrido sobre Fable 5.1 el 24 % (tabla 10). Esas ganancias eran un artefacto del diseño. El inicio fijo caía dentro de la ráfaga de alarmas de arranque del detector y justo después del calentamiento de 8 h de la memoria de los planificadores, cuando C2 repara muchas estaciones a la vez; las reparaciones rutinarias coincidían entonces con el fallo o lo precedían por minutos y, con la regla de reparación antigua, lo eliminaban sin coste. Sacar los inicios del periodo de arranque, permitir que las reparaciones eliminen solo fallos activos y limitar la métrica a 24 h eliminó esas ganancias; la clasificación de los dos diseños LLM se invirtió solo con el detector por estación (con el detector

Tabla 10: Los mismos planificadores con el primer diseño de evaluación (v1: fallo a 600 min, una reparación elimina los fallos que empiezan durante ella, recuperación sobre toda la traza, 72 escenarios) y con el actual (v2: tres inicios aleatorios después del periodo de arranque, una reparación elimina solo los fallos activos, recuperación neta a 24 h, 216 escenarios). Trazas reservadas.

Planificador	v1, heredado	v2, heredado	v2, por estación
Adaptativa + veto por carga	+0,54	−0,02	+0,20
C2 (RAG), Qwen 7B	+0,20	−0,15	−0,25
Híbrido, Qwen 7B	+0,41	−0,04	+0,29
C2 (RAG), Fable 5.1	+0,58	+0,07	+0,14
Híbrido, Fable 5.1	+0,24	+0,03	+0,41

heredado, C2 sobre Fable 5.1 sigue por delante del híbrido, +0,07 frente a +0,03). Presentamos el primer diseño porque es un error fácil de cometer: un controlador en bucle cerrado evaluado en un único instante de fallo fijo hereda lo que le ocurra a la línea en ese instante.

8. Discusión

La capa Analyze y el planificador interactúan. Cerrar el bucle original [3] sobre su propio detector no recupera casi nada en un día: ningún planificador recupera más de 0,07, porque el detector informa de un fallo tarde, pocas veces y dentro de ráfagas de toda la línea. Sustituirlo por un test por estación sobre los tiempos de proceso lleva a la mejor regla de −0,02 a +0,20 y al mejor planificador de +0,07 a +0,41. Pero el planificador importa tanto como el detector una vez que las alarmas son útiles: con el detector por estación, los planificadores abarcan 0,66, de −0,25 a +0,41. La capa Analyze fija lo que un planificador puede conseguir; el planificador decide cuánto de ello se consigue.

Proponer y vetar: el valor está en no actuar. Con alarmas útiles, el LLM aporta más como veto sobre un test estadístico, y lo hace declinando actuar, no detectando más. La descomposición del retraso muestra el efecto: la regla elimina más parte del fallo, pero sus reparaciones falsas cuestan más que el fallo adicional que elimina; el híbrido deja más parte del fallo y paga un tercio del coste. Buena parte de esto es un argumento sencillo que una regla puede implementar: no reparar una estación cuya alarma comparten muchas otras. Una regla que solo hace eso, con el veto por carga, reproduce entre dos tercios y tres cuartos del margen del híbrido sobre la regla adaptativa (de +0,20 a +0,34 en las trazas reservadas, de +0,17 a +0,40 en las de validación). Lo

que el LLM añade sobre ella, +0,08 y +0,09, viene de aplicar el argumento de forma selectiva: en todos los tamaños de alarma veta la estación averiada con menos frecuencia que una sana (tabla 9). Declinar no basta por sí solo: vetar al azar no ayuda, y C1 sobre el modelo de frontera, sin el test ni conocimiento recuperado, apenas actúa y recupera poco. Por sí solos, los dos planificadores LLM lo hacen peor que la regla.

El veto vale lo que cuesta una reparación falsa. El análisis de sensibilidad da al resultado su alcance (tabla 7). Donde las reparaciones falsas son caras (fallos leves, reparaciones largas, línea parada, congestión completa), el híbrido sobre Fable 5.1 es con diferencia el mejor planificador, a menudo el único que no hace mucho más daño que bien. Donde actuar es barato (fallos graves, reparaciones cortas, sin congestión), todo planificador razonable recupera alrededor de la mitad del retraso y el LLM no añade nada. Y cuando los fallos sí tienen una causa común, el argumento de las ráfagas los descarta: el híbrido queda por detrás de la regla simple, y el híbrido de 7B, que no usa el argumento, es el mejor. El valor de un veto con LLM no es, por tanto, una propiedad del modelo sino de la planta: depende del coste de una intervención innecesaria y de si, en ella, las alarmas compartidas por varias estaciones suelen ser ruido o suelen ser un fallo común.

El modelo importa dentro de la arquitectura adecuada. El mismo híbrido recupera +0,41 sobre Fable 5.1 y +0,29 sobre el modelo de 7B en las trazas reservadas, y +0,49 frente a +0,23 en las trazas nuevas, donde el híbrido de 7B cae por debajo de las mejores reglas. Solo la ventaja del modelo de frontera se replica. Es un contraste entre dos sistemas, que difieren en tamaño, cuantización, proveedor y ajustes de inferencia, no una curva de capacidad medida.

Implicaciones para la práctica. Dentro de los límites de un estudio con fallos sintéticos, cuatro puntos son trasladables a arquitectos de automatización e ingenieros de mantenimiento. Primero, conectar alarmas de umbral directamente a órdenes de mantenimiento es perjudicial: el planificador ingenuo es peor que no hacer nada con los dos detectores, porque las reparaciones falsas se acumulan; las alarmas deben racionalizarse antes de que desencadenen acciones, por analogía con lo que la gestión de alarmas exige para los operadores [13, 14]. Segundo, un detector debe comparar cada estación con su propio comportamiento reciente en la parte de la señal que afecta un fallo, aquí los tiempos de proceso y no transiciones dominadas por el almacenamiento, y con un test que controle sus falsas alarmas. Tercero, que actuar compense depende de la gravedad de los fallos en relación con el coste de

una reparación y de la capacidad de respaldo durante las reparaciones; antes de añadir un LLM, una planta debería probar una regla que suprima las acciones sobre alarmas compartidas por muchas estaciones, que aquí recupera la mayor parte de lo que hace el LLM, y comprobar con qué frecuencia sus alarmas compartidas son en realidad fallos de causa común. Cuarto, un veto con LLM compensa su coste donde las intervenciones innecesarias son caras; su no determinismo, su sensibilidad a cómo se estructuran las alarmas y su fallo con causas comunes aconsejan mantener toda acción detrás de un validador simbólico, como aquí, y, en una planta, detrás de una aprobación humana en el sistema de gestión del mantenimiento que emite las órdenes de trabajo.

Los conjuntos de desarrollo engañan. El detector por estación y las reglas reajustadas se eligieron en la traza de desarrollo, donde el detector nombra a la estación averiada en los 72 escenarios y la regla adaptativa recupera +0,84; en las trazas reservadas el detector la nombra en 122 de 216 y la misma regla recupera +0,20, y las configuraciones reajustadas allí ayudan a una regla y no a las demás. Una sola traza es demasiado pequeña para seleccionar diseños, y los controladores basados en LLM deberían evaluarse sobre datos no usados para escribir sus prompts, con inicios de fallo que no coincidan con el arranque del controlador.

Amenazas a la validez. De constructo: los fallos son sintéticos y el modelo de efectos es aditivo y guiado por la traza, sin colas, de modo que toda recuperación es una propiedad del modelo, no una propiedad medida de la línea de Bosch; sus parámetros se variaron de uno en uno para todos los planificadores (tabla 7), pero no de forma conjunta, y el prompt congelado describe reparaciones de 60 min con capacidad de respaldo en todas las condiciones. Los fallos de causa común se probaron en una sola forma (un fallo en cuatro estaciones a la vez, 54 escenarios). La recuperación es neta del coste de actuar, medida en tiempo de producción en las 24 h siguientes al fallo; un coste fijo por reparación, que probamos como comprobación, solo amplía la ventaja del híbrido, pero no usamos pesos económicos. Una ventana de 24 h es tiempo cronológico, no productivo, y en alrededor de un tercio de los escenarios el fallo no cuesta nada en ella (tabla 4). La referencia del detector por estación son las 24 h anteriores de la estación, de modo que un fallo persistente queda absorbido en ella al cabo de un día. Las gemelas son estructurales, no equivalencias de equipo verificadas. *Interna*: las trazas reservadas informaron la decisión de construir un detector nuevo y el diseño de evaluación, aunque no sus parámetros (sección 6); las

trazas de validación se generaron después de congelarlo todo. Cada decisión del modelo de frontera es una muestra almacenada con esfuerzo bajo, y la mayoría de las llamadas reutiliza una respuesta almacenada, lo que además hace que los escenarios compartan decisiones; tres réplicas sin las respuestas almacenadas cambiaron la recuperación principal en 0,014 como mucho. El prompt v3 se probó en la traza de desarrollo solo con el modelo de 7B; los prompts forman parte del tratamiento; las reglas de control cubren el veto por carga y el argumento de las ráfagas, pero no todas las estadísticas que recupera C2, como el resultado de reparaciones anteriores; el modelo de 7B está cuantizado a 4 bits. El modelo de frontera puede haber visto nuestros artículos anteriores [3, 4], que nombran la línea de Bosch, pero los fallos son sintéticos y no están descritos en ningún sitio que haya podido ver. *Externa*: una línea de producción, un conjunto de datos, solo las piezas del conjunto de prueba; dos modelos de lenguaje, sin tamaños intermedios. *Estadística*: las ventanas de 24 h de los escenarios de una misma traza se solapan; el conjunto reservado tiene tres trazas y el de validación cinco, y remuestrear trazas o grupos de traza y estación solo aborda en parte la dependencia.

9. Conclusión

Cerramos el bucle autonómico de una arquitectura de fabricación autocoordinada que, tal como se publicó, solo detectaba. Con su propio detector, ningún planificador recupera retraso: el detector emite ráfagas de alarmas de toda la línea e informa de una estación averiada tarde y pocas veces. Con un test de cambio por estación sobre los tiempos de proceso, y dentro de nuestro modelo de efectos, un LLM de frontera con estadísticas operativas recuperadas que veta los candidatos del test recupera alrededor de dos quintas partes del retraso en un día en las trazas reservadas y la mitad en cinco trazas generadas después de congelar el diseño, por delante de todas las reglas que probamos, y tres réplicas lo reproducen. Pero una regla simple que veta las alarmas compartidas por muchas estaciones recorre la mayor parte del camino, y el margen restante del LLM, de 8–9 puntos, viene de aplicar ese argumento de forma selectiva. El veto vale mucho donde las reparaciones falsas son caras, nada donde actuar es barato, y perjudica cuando los fallos tienen una causa común. Usado por sí solo, el mismo modelo no supera al test, y la ventaja del modelo de 7B no se replica. Una evaluación anterior, con un instante de fallo fijo, había sugerido las conclusiones contrarias. El trabajo futuro validará el modelo de efectos con datos de planta con capacidades, tiempos de reparación y fallos de causa común reales, añadirá tamaños de modelo intermedios y probará el detector con fallos que

persisten más de un día.

Apéndice A. Prompts

Prompt de sistema v3 de C1, tal como se envía a ambos modelos (exportado del código):

```
You are the Plan phase of an autonomic controller for a manufacturing line
     (the Bosch production line: about 50 stations on 4 sub-lines).

How alarms work. The Analyze phase checks each station separately. It
    raises an alarm listing the stations whose recent processing times (
    transitions of up to 60 minutes leaving the station) exceed the
    station's own recent history more often than chance would explain. In
    normal operation this detector also raises false alarms: a station
    named in an alarm may be perfectly healthy.

What you can do for each flagged station:
- "none": leave it alone;
- "maintain": repair it for 60 minutes. While it is being repaired a
    slower backup handles its work. Repairing a degraded station fixes it.
     Repairing a healthy station fixes nothing and only slows production;
- "reroute_and_maintain": only for a station with a free twin. The twin
    takes all of the station's load during the repair (the twin slows down
     because of the extra load), and the station is repaired.

Objective: minimise total production delay. Acting on a healthy station
    wastes production. Ignoring a truly degraded station lets its delay
    accumulate for as long as it stays degraded.

For EVERY station listed, first give a verdict, then an action:
- "degraded": the evidence shows the station itself is not working
    properly;
- "normal": the station is probably healthy and the alarm is a false alarm
    ;
- "self_inflicted": the station is alarming because of load you rerouted
    onto it;
- "insufficient_evidence": you cannot tell.
Only a "degraded" station may receive an action other than "none".

Answer only with JSON matching the schema. Keep "reason" under 30 words.
```

C2 añade:

```
You also receive RETRIEVED KNOWLEDGE for each station, computed from your
    operational log:
- how many alarms the station raised recently, and how many its normal (
    fault-free) alarm rate would predict over the same period;
```

```
- whether you are currently pushing extra load onto it;
- what happened to its alarms after you repaired it before;
- its recent log records.
How to read it:
- many more alarms than expected, sustained over the last 24 h, points to
    "degraded";
- about as many alarms as expected points to "normal";
- a station receiving load you rerouted onto it is "self_inflicted":
    repairing it does not help;
- if alarms did not drop after a previous repair, the alarms were probably
     not caused by a fault;
- with too little history, prefer "insufficient_evidence".
```

El híbrido añade, tras el texto de C2:

```
In this mode a statistical change detector has already screened the alarms
    . The stations listed were flagged because their recent alarm count is
     improbably high compared with their own recent history. The detector
    is usually right, but it cannot see context: load you rerouted onto a
    station, a previous repair that did not reduce its alarms, a burst of
    alarms shared by many stations at once. Give "degraded" unless the
    retrieved knowledge gives a concrete reason to doubt the detector;
    then give the verdict that reason supports.
```

El prompt v2, usado con el detector heredado, sustituye la primera frase del segundo párrafo del prompt de C1 (de “How alarms work” a “explain.”) por:

```
How alarms work. The Analyze phase raises an alarm listing stations whose
    recent transition times exceeded 1.3 times their reference time within
     the same batch of events.
```

El mensaje de usuario indica el instante, las estaciones que hay que juzgar con el estado de sus gemelas, las estaciones en reparación y los desvíos activos; para C2 y el híbrido añade, por estación, el conocimiento recuperado descrito en la sección 5. El esquema JSON de la respuesta está en el repositorio.

Disponibilidad de datos y código

El simulador sin interfaz gráfica, los planificadores, los prompts, el registro de desarrollo, cada decisión de los modelos con la razón que dieron, los resultados y los scripts que producen cada tabla y figura están disponibles en `https://github.com/mbsanchez/ssacsimulator`. El simulador usa solo la biblioteca estándar de Python; la estadística y las figuras usan además SciPy y matplotlib. El conjunto de datos de Bosch es público [18]; los ficheros derivados de él (modelo de proceso y trazas de eventos) no se

redistribuyen, y el repositorio incluye los scripts que los producen, entre ellos el port a Python 3 del preprocesamiento original usado para las cinco trazas de validación, y sus sumas de verificación.

Declaración de contribución de autoría (CRediT)

Manuel Sánchez: conceptualización, metodología, investigación, visualización, redacción – borrador original. **Ernesto Exposito:** supervisión, redacción – revisión y edición.

Declaración de conflicto de intereses

Los autores declaran no tener intereses financieros en conflicto ni relaciones personales conocidas que pudieran haber influido en el trabajo presentado en este artículo.

Declaración sobre el uso de IA generativa y tecnologías asistidas por IA en la preparación del manuscrito

Durante la preparación de este trabajo los autores usaron Claude (Anthropic) para asistir en el desarrollo del software de simulación y análisis, redactar y revisar el texto del manuscrito, y como agente adversario que actuó como revisor para identificar debilidades y sugerir mejoras. El software fue revisado por el primer autor. Tras usar esta herramienta, los autores revisaron y editaron el contenido según fue necesario y asumen toda la responsabilidad sobre el contenido del artículo publicado. El uso de Qwen2.5 y Claude Fable 5.1 como planificadores forma parte del método estudiado en este artículo (sección 5) y no guarda relación con la preparación del manuscrito.

Referencias

[1] J. O. Kephart, D. M. Chess, The vision of autonomic computing, Computer 36 (1) (2003) 41–50. doi:10.1109/MC.2003.1160055.

[2] D. Weyns, An Introduction to Self-adaptive Systems: A Contemporary Software Engineering Perspective, Wiley-IEEE Press, 2021. doi:10.1002/9781119574910.

[3] M. Sánchez, E. Exposito, J. Aguilar, Implementing self-* autonomic properties in self-coordinated manufacturing processes for the Industry 4.0 context, Computers in Industry 121 (2020) 103247. doi:10.1016/j.compind.2020.103247.

[4] M. Sánchez, E. Exposito, J. Aguilar, Autonomic computing in manufacturing process coordination in Industry 4.0 context, Journal of Industrial Information Integration 19 (2020) 100159. doi:10.1016/j.jii.2020.100159.

[5] Y. Xia, M. Shenoy, N. Jazdi, M. Weyrich, Towards autonomous system: flexible modular production system enhanced with large language model agents, in: 2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA), 2023, pp. 1–8. doi:10.1109/ETFA54631.2023.10275362.

[6] M. Li, Q. Zhou, W. Li, T. Qu, M. Yang, P. Jiang, A4PS: Agentic AI-assisted advanced planning and scheduling with large language models for smart manufacturing, Journal of Manufacturing Systems 85 (2026) 207–226. doi:10.1016/j.jmsy.2026.01.003.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 9459–9474.

[8] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, K. Tei, Generative AI for self-adaptive systems: State of the art and research roadmap, ACM Transactions on Autonomous and Adaptive Systems 19 (3) (2024) 13:1–13:60. doi:10.1145/3686803.

[9] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, K. Tei, Exploring the potential of large language models in self-adaptive systems, in: Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), 2024, pp. 77–83. doi:10.1145/3643915.3644088.

[10] N. Nascimento, P. Alencar, D. Cowan, Self-adaptive large language model (LLM)-based multiagent systems, in: 2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C), 2023, pp. 104–109. doi:10.1109/ACSOS-C58168.2023.00048.

[11] L. Alves, G. L. B. De Oliveira, M. Chagas, A. Bandeira, P. H. Maia, C. Rocha, MAPER: Extending MAPE-K with LLM-based reasoning to manage unanticipated situations in self-adaptive systems, in: Proceedings of the 21st International Conference on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), 2026, pp. 239–250. doi:10.1145/3788550.3794886.

[12] M. Sánchez, E. Exposito, J. Aguilar, Industry 4.0: survey from a system integration perspective, International Journal of Computer Integrated Manufacturing 33 (10–11) (2020) 1017–1041. doi:10.1080/0951192X.2020.1775295.

[13] International Society of Automation, ANSI/ISA-18.2-2016: Management of Alarm Systems for the Process Industries, International Society of Automation, Research Triangle Park, NC, american National Standard (2016).

[14] International Society of Automation, ISA-TR18.2.6-2012: Alarm systems for batch and discrete processes, Tech. rep., International Society of Automation, Research Triangle Park, NC (2012).

[15] M. Basseville, I. V. Nikiforov, Detection of Abrupt Changes: Theory and Application, Prentice Hall, 1993.

[16] N. D. Thuan, L. H. Anh, Large language models for fault diagnosis in smart manufacturing: A comprehensive review, Intelligent Systems with Applications 31 (2026) 200714. doi:10.1016/j.iswa.2026.200714.

[17] M. Sadeqi Bajestani, D. Mun, D. B. Kim, Human-in-the-loop and large language models in smart manufacturing: Current applications, challenges, and perspectives, Journal of Manufacturing Systems 86 (2026) 913–941. doi:10.1016/j.jmsy.2026.04.027.

[18] Bosch, Bosch production line performance, Kaggle competition dataset, https://www.kaggle.com/c/bosch-production-line-performance (2016).

[19] Qwen Team, Qwen2.5 technical report, arXiv:2412.15115 (2024).

[20] Ollama, Ollama: run large language models locally, https://ollama.com, version 0.35.1 (2026).

[21] Anthropic, Claude agent sdk, https://code.claude.com/docs/en/agent-sdk, python package claude-agent-sdk 0.2.163 (2026).

[22] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[23] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[24] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.